Computational Linear Algebra: PCA Homework

Exercise 0.0: Initialization

Fill the missing values in this text box. **Remove the information of the second student if you are not working in a team**.

Academic Year: 2025/2026

Team Members (Alphabetical Order):

1. Fabi, Pietro ( ).

Now, fill the python list below with the Student IDs of the team

In [1]:
```
StudentIDs = [      ]  # <-------- Fill this list with Student IDs of the team
```

Exercise 0.1: Starting Code-Cell

Download the .csv files from the web page of the course (*responses_hw.csv* and *columns_hw.csv*) and past them in **the same folder of this notebook**.

Then, run the cell below, **without modifing any line of code**.

The output of this code cell is **your personal subset of the original dataset**, with 2/3 of the original features (i.e., columns) and 3/4 of the original persons (i.e., rows).

ATTENTION: DO NOT CHANGE THE CODE INSIDE THE FOLLOWING CELL, ANY CHANGE CAN INVALIDATE THE HOMEWORK!

In [2]:
```
########################################################################
############## DO NOT CHANGE THE CODE IN THIS CELL #################
########################################################################

import numpy as np
import pandas as pd
from IPython.display import display

var_entertainment_feat_types = ['Interests', 'Movies', 'Music']
var_personal_feat_types = ['Finance', 'Phobias']
fixed_feat_types = ['Personality', 'Health']

label_types = ['Demographic']

variables_by_type = {
    'Demographics': ['Age', 'Height', 'Weight', 'Number of siblings',
```

```
                    'Gender', 'Hand', 'Education', 'Only child', 'Home Town Typ
                    'Home Type'],
    'Finance': ['Finances', 'Shopping centres', 'Branded clothing',
                'Entertainment spending', 'Spending on looks',
                'Spending on gadgets', 'Spending on healthy eating'],
    'Health': ['Smoking', 'Alcohol', 'Healthy eating'],
    'Interests': ['History', 'Psychology', 'Politics', 'Mathematics',
                  'Physics', 'Internet', 'PC', 'Economy Management',
                  'Biology', 'Chemistry', 'Reading', 'Geography',
                  'Foreign languages', 'Medicine', 'Law', 'Cars',
                  'Art exhibitions', 'Religion', 'Countryside, outdoors',
                  'Dancing', 'Musical instruments', 'Writing', 'Passive sport',
                  'Active sport', 'Gardening', 'Celebrities', 'Shopping',
                  'Science and technology', 'Theatre', 'Fun with friends',
                  'Adrenaline sports', 'Pets'],
    'Movies': ['Movies', 'Horror', 'Thriller', 'Comedy', 'Romantic',
               'Sci-fi', 'War', 'Fantasy/Fairy tales', 'Animated',
               'Documentary', 'Western', 'Action'],
    'Music': ['Music', 'Slow songs or fast songs', 'Dance', 'Folk',
              'Country', 'Classical music', 'Musical', 'Pop', 'Rock',
              'Metal or Hardrock', 'Punk', 'Hiphop, Rap', 'Reggae, Ska',
              'Swing, Jazz', 'Rock n roll', 'Alternative', 'Latino',
              'Techno, Trance', 'Opera'],
    'Personality': ['Daily events', 'Prioritising workload',
                    'Writing notes', 'Workaholism', 'Thinking ahead',
                    'Final judgement', 'Reliability', 'Keeping promises',
                    'Loss of interest', 'Friends versus money', 'Funniness',
                    'Fake', 'Criminal damage', 'Decision making', 'Elections',
                    'Self-criticism', 'Judgment calls', 'Hypochondria',
                    'Empathy', 'Eating to survive', 'Giving',
                    'Compassion to animals', 'Borrowed stuff',
                    'Loneliness', 'Cheating in school', 'Health',
                    'Changing the past', 'God', 'Dreams', 'Charity',
                    'Number of friends', 'Punctuality', 'Lying', 'Waiting',
                    'New environment', 'Mood swings', 'Appearence and gestures',
                    'Socializing', 'Achievements', 'Responding to a serious lett
                    'Children', 'Assertiveness', 'Getting angry',
                    'Knowing the right people', 'Public speaking',
                    'Unpopularity', 'Life struggles', 'Happiness in life',
                    'Energy levels', 'Small - big dogs', 'Personality',
                    'Finding lost valuables', 'Getting up', 'Interests or hobbie
                    "Parents' advice", 'Questionnaires or polls', 'Internet usag
    'Phobias': ['Flying', 'Storm', 'Darkness', 'Heights', 'Spiders', 'Snakes',
                'Rats', 'Ageing', 'Dangerous dogs', 'Fear of public speaking']
}

labels = variables_by_type['Demographics']
features_all = []
for tt in variables_by_type.keys():
    if tt != 'Demographics':
        features_all += variables_by_type[tt]

def which_features(*StudentIDs):
    random_seed = min(StudentIDs)
    np.random.seed(random_seed)
    features_ = np.random.choice(features_all, int((2 * len(features_all)) / 3),
    features = []
    features_by_type = {tt: [] for tt in variables_by_type.keys() if tt != 'Demo
    for tt in variables_by_type.keys():
        ft_list = variables_by_type[tt]
```

```
        for ii in range(len(ft_list)):
            if ft_list[ii] in features_:
                features.append(ft_list[ii])
                features_by_type[tt].append(ft_list[ii])

    return features, features_by_type

features, features_by_type = which_features(*StudentIDs)

print(f'*** THESE ARE THE {len(features)} SELECTED FEATURES (SEE VARIABLE featur
for ff in features:
    print(f'{ff}')
print('************************************')
print('')
print('*** SELECTED FEATURES BY TYPES (SEE VARIABLE features_by_type):')
for tt in features_by_type.keys():
    print(f'{tt}: {features_by_type[tt]}')
    print('')
print('************************************')
print('')
print('*** THESE ARE THE LABELS (SEE VARIABLE labels):')
for ll in labels:
    print(f'{ll}')
print('************************************')

def which_rows(df, frac, *StudentIDs):
    random_seed = min(StudentIDs)
    df_ = df.sample(frac=frac, random_state=random_seed)
    return df_

responses_hw = pd.read_csv('responses_hw.csv', index_col=0)
responses = which_rows(responses_hw, 0.75, *StudentIDs)
responses = responses.loc[:, features + labels]

responses_ft = responses.loc[:, features]
responses_lb = responses.loc[:, labels]

print('')
print('*** THIS IS YOUR PERSONAL DATASET (features AND labels TOGETHER, SEE VARI
display(responses)
print('')
print('*** THIS IS YOUR PERSONAL DATASET (features, SEE VARIABLE responses_ft)')
display(responses_ft)
print('')
print('*** THIS IS YOUR PERSONAL DATASET (labels, SEE VARIABLE responses_lb)')
display(responses_lb)

random_seed = min(StudentIDs)
np.random.seed(random_seed)

your_scaler = np.random.choice(['StandardScaler', 'MinMaxScaler'])
```

*** THESE ARE THE 93 SELECTED FEATURES (SEE VARIABLE features):
Finances
Shopping centres
Branded clothing
Entertainment spending
Spending on looks
Spending on healthy eating
Smoking
Psychology
Politics
Mathematics
Internet
PC
Chemistry
Reading
Foreign languages
Medicine
Law
Art exhibitions
Religion
Countryside, outdoors
Writing
Celebrities
Science and technology
Theatre
Fun with friends
Adrenaline sports
Pets
Movies
Thriller
Comedy
Sci-fi
Fantasy/Fairy tales
Animated
Documentary
Action
Slow songs or fast songs
Dance
Folk
Classical music
Musical
Pop
Rock
Punk
Rock n roll
Techno, Trance
Opera
Daily events
Prioritising workload
Writing notes
Final judgement
Reliability
Keeping promises
Loss of interest
Funniness
Fake
Criminal damage
Decision making
Elections
Self-criticism

Judgment calls
Hypochondria
Empathy
Eating to survive
Giving
Loneliness
Health
Changing the past
God
Dreams
Charity
Waiting
New environment
Appearence and gestures
Responding to a serious letter
Children
Getting angry
Public speaking
Unpopularity
Happiness in life
Energy levels
Small - big dogs
Finding lost valuables
Getting up
Interests or hobbies
Parents' advice
Internet usage
Storm
Darkness
Spiders
Snakes
Ageing
Dangerous dogs
Fear of public speaking
****

*** SELECTED FEATURES BY TYPES (SEE VARIABLE features_by_type):
Finance: ['Finances', 'Shopping centres', 'Branded clothing', 'Entertainment spen
ding', 'Spending on looks', 'Spending on healthy eating']

Health: ['Smoking']

Interests: ['Psychology', 'Politics', 'Mathematics', 'Internet', 'PC', 'Chemistr
y', 'Reading', 'Foreign languages', 'Medicine', 'Law', 'Art exhibitions', 'Religi
on', 'Countryside, outdoors', 'Writing', 'Celebrities', 'Science and technology',
'Theatre', 'Fun with friends', 'Adrenaline sports', 'Pets']

Movies: ['Movies', 'Thriller', 'Comedy', 'Sci-fi', 'Fantasy/Fairy tales', 'Animat
ed', 'Documentary', 'Action']

Music: ['Slow songs or fast songs', 'Dance', 'Folk', 'Classical music', 'Musica
l', 'Pop', 'Rock', 'Punk', 'Rock n roll', 'Techno, Trance', 'Opera']

Personality: ['Daily events', 'Prioritising workload', 'Writing notes', 'Final ju
dgement', 'Reliability', 'Keeping promises', 'Loss of interest', 'Funniness', 'Fa
ke', 'Criminal damage', 'Decision making', 'Elections', 'Self-criticism', 'Judgme
nt calls', 'Hypochondria', 'Empathy', 'Eating to survive', 'Giving', 'Lonelines
s', 'Health', 'Changing the past', 'God', 'Dreams', 'Charity', 'Waiting', 'New en
vironment', 'Appearence and gestures', 'Responding to a serious letter', 'Childre
n', 'Getting angry', 'Public speaking', 'Unpopularity', 'Happiness in life', 'Ene

rgy levels', 'Small - big dogs', 'Finding lost valuables', 'Getting up', 'Interes
ts or hobbies', "Parents' advice", 'Internet usage']

Phobias: ['Storm', 'Darkness', 'Spiders', 'Snakes', 'Ageing', 'Dangerous dogs',
'Fear of public speaking']

****

*** THESE ARE THE LABELS (SEE VARIABLE labels):
Age
Height
Weight
Number of siblings
Gender
Hand
Education
Only child
Home Town Type
Home Type
****

*** THIS IS YOUR PERSONAL DATASET (features AND labels TOGETHER, SEE VARIABLE res
ponses)

	Finances	Shopping centres	Branded clothing	Entertainment spending	Spending on looks	Spending on healthy eating	Smoking	Psych
396	1	5	5	5	5	5	never smoked	
254	4	5	5	4	5	4	former smoker	
395	3	3	2	3	2	2	tried smoking	
95	3	4	2	4	4	4	never smoked	
307	5	4	3	4	3	4	tried smoking	
...	...	...	...	...	...	...	...	
208	1	3	4	4	4	5	tried smoking	
171	3	5	3	4	4	3	former smoker	
531	4	4	1	4	4	2	never smoked	
56	4	3	2	2	2	5	tried smoking	
252	3	3	4	3	4	3	tried smoking	

506 rows × 103 columns

```
*** THIS IS YOUR PERSONAL DATASET (features, SEE VARIABLE responses_ft)
```

	Finances	Shopping centres	Branded clothing	Entertainment spending	Spending on looks	Spending on healthy eating	Smoking	Psych
396	1	5	5	5	5	5	never smoked	
254	4	5	5	4	5	4	former smoker	
395	3	3	2	3	2	2	tried smoking	
95	3	4	2	4	4	4	never smoked	
307	5	4	3	4	3	4	tried smoking	
...	...	...	...	...	...	...	...	
208	1	3	4	4	4	5	tried smoking	
171	3	5	3	4	4	3	former smoker	
531	4	4	1	4	4	2	never smoked	
56	4	3	2	2	2	5	tried smoking	
252	3	3	4	3	4	3	tried smoking	

506 rows × 93 columns

```
*** THIS IS YOUR PERSONAL DATASET (labels, SEE VARIABLE responses_lb)
```

	Age	Height	Weight	Number of siblings	Gender	Hand	Education	Only child	Home Town Type	I
396	17	183	75	3	male	right	primary school	no	city	b
254	20	161	59	1	female	right	secondary school	no	city	b
395	20	165	48	1	female	right	college/bachelor degree	no	city	hous
95	20	157	54	1	female	right	secondary school	no	city	b
307	19	185	78	2	male	left	secondary school	no	village	hous
...	...	...	...	...	...	...	...	...	...	
208	20	173	58	1	female	right	secondary school	no	city	hous
171	18	173	64	3	female	right	secondary school	no	city	hous
531	17	170	55	3	female	right	secondary school	no	village	hous
56	20	175	83	1	male	left	secondary school	yes	city	b
252	19	189	76	1	male	right	secondary school	no	city	b

506 rows × 10 columns

Exercise 0.2: Importing Modules

In the following cell, import all the modules you think are necessary for doing the homework, **among the ones listed and used during the laboratories of the course**.

For reproducibility, **no extra modules are allowed**.

DO NOT IMPORT NUMPY NOR PANDAS, they are already imported.

In [3]:
```
# DO NOT IMPORT NUMPY AND PANDAS - Already imported
%matplotlib widget
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score, silhouette_samples
from matplotlib.lines import Line2D
import matplotlib.cm as cm
```

Exercise 1. Dataset Preprocessing

In this exercise, you have to do the following operations on the dataset of the features **responses_ft**:

1. Create a new dataframe called **responses_ft_enc** by encoding the categorical features (if they exist), motivating your choices;
2. Create a new dataframe called **responses_ft_pp** by preprocessing the data in **responses_ft_enc**, according to the scaler reported in the cell below.

In [4]:
```
print(f'*** YOU HAVE TO APPLY A PREPROCESSING USING THE {your_scaler}')
```

```
*** YOU HAVE TO APPLY A PREPROCESSING USING THE StandardScaler
```

For doing this exercise, fill the cells below following the instructions you read.

Describe and motivate the encoding operations you will apply (max 150 words):

The majority of the features are numerical, except for 'Smoking' and 'Internet usage', which are categorical. Since these variables possess an intrinsic natural order for example, 'few hours a day' represents a lower usage intensity than 'most of the day', we will apply Ordinal Encoding. This transformation maps the categories to incremental integers 0, 1, 2..., preserving the hierarchical relationship of the data while converting it into a numeric format required for the analysis.

Describe the preprocessing operation you will apply and comment the effects it may have on the data (max 150 words):

We will first apply the Ordinal Encoder to the categorical variables to ensure the entire dataset is numeric. Subsequently, we will apply the StandardScaler to normalize all features to a mean of 0 and a standard deviation of 1

Write the code for performing the encoding and preprocessing operations of the exercise. Show the encoded data and the preprocessed data you obtain, plus any additional table/value that can be useful for commenting the results:

In [5]:
```
responses_ft_enc = responses_ft.copy()

categories = [['never smoked', 'tried smoking', 'former smoker', 'current smoker
              ['less than an hour a day', 'few hours a day', 'most of the day']]

oe = OrdinalEncoder(categories=categories, dtype=np.int64)

cols_to_transform = ["Smoking", "Internet usage"]
responses_ft_enc[cols_to_transform] = oe.fit_transform(responses_ft[cols_to_tran

ss = StandardScaler()
responses_ft_pp = ss.fit_transform(responses_ft_enc)
responses_ft_pp = pd.DataFrame(responses_ft_pp, columns=responses_ft.columns.val

print('*** THIS IS THE ENCODED DATASET')
```

```
display(responses_ft_enc)

print('*** THIS IS PREPROCESSED DATASET')
display(responses_ft_pp)
```

*** THIS IS THE ENCODED DATASET

	Finances	Shopping centres	Branded clothing	Entertainment spending	Spending on looks	Spending on healthy eating	Smoking	Psych
396	1	5	5	5	5	5	0	
254	4	5	5	4	5	4	2	
395	3	3	2	3	2	2	1	
95	3	4	2	4	4	4	0	
307	5	4	3	4	3	4	1	
...	...	...	...	...	...	...	...	
208	1	3	4	4	4	5	1	
171	3	5	3	4	4	3	2	
531	4	4	1	4	4	2	0	
56	4	3	2	2	2	5	1	
252	3	3	4	3	4	3	1	

506 rows × 93 columns

*** THIS IS PREPROCESSED DATASET

	Finances	Shopping centres	Branded clothing	Entertainment spending	Spending on looks	Spending on healthy eating	Smoking	Psy
396	-1.735047	1.320710	1.523897	1.517613	1.562205	1.414114	-1.352603	-1
254	0.906679	1.320710	1.523897	0.678365	1.562205	0.489642	0.648617	0
395	0.026104	-0.205042	-0.748476	-0.160884	-0.905461	-1.359304	-0.351993	1
95	0.026104	0.557834	-0.748476	0.678365	0.739650	0.489642	-1.352603	0
307	1.787255	0.557834	0.008982	0.678365	-0.082906	0.489642	-0.351993	-1
...	...	...	...	...	...	...	...	
208	-1.735047	-0.205042	0.766440	0.678365	0.739650	1.414114	-0.351993	-0
171	0.026104	1.320710	0.008982	0.678365	0.739650	-0.434831	0.648617	-0
531	0.906679	0.557834	-1.505934	0.678365	0.739650	-1.359304	-1.352603	1
56	0.906679	-0.205042	-0.748476	-1.000132	-0.905461	1.414114	-0.351993	-1
252	0.026104	-0.205042	0.766440	-0.160884	0.739650	-0.434831	-0.351993	-1

506 rows × 93 columns

Comment the results obtained after the preprocessing operation (max 100 words):

Our preprocessing successfully transformed the dataset into a homogeneous numerical matrix suitable for our purpose. The Ordinal Encoding mapped the categorical variables to integers and then the StandardScaler normalized all the features. This transformation is crucial as it removes scale bias, ensuring that features with originally larger ranges do not disproportionately dominate the variance calculations in PCA or the Euclidean distance metrics in clustering.

Exercise 2. Analyzing the Variance and the PCs

In this exercise, you have to do the following operations:

1. compute and visualize the variance of all the features in *responses_ft_enc* and *responses_ft_pp*;
2. compute all the n Principal Components (PCs) for *responses_ft_enc* and *responses_ft_pp*, separately, and visualize the curves of the cumulative explained variances.

For doing this exercise, fill the cells below following the instructions you read.

Write the code for computing and visualizing the variance of the features of the two datasets:

In [9]:

```
def create_colors_and_legend(ft_names, ft_category):
    legend_created = []
    category_colors = {
    'Finance': '#FFFF00',
    'Health': '#00FF00',
    'Interests': '#00FFFF',
    'Movies': '#800080',
    'Music': '#FFC0CB',
    'Personality': '#FFA500',
    'Phobias': '#000000'}

    for key in category_colors.keys():
        legend_created.append(Line2D([0], [0], marker='s', color='w', label=key,

    feature_to_category_map = {}
    for cat, it in ft_category.items():
        for i in it:
            feature_to_category_map[i] = cat

    bar_colors = []
    for feature in ft_names:
        cat = feature_to_category_map.get(feature)
        color = category_colors.get(cat, '#CCCCCC')
        bar_colors.append(color)

    return bar_colors, legend_created

def variance_graph(values, positions, names, word, colors_map, legend):
    fig_var = plt.figure(figsize=(11, 10))
    plt.barh(positions, values, color=colors_map, edgecolor='black', alpha=0.7)
    plt.yticks(positions, names, fontsize=8)
    plt.ylim(-1, len(names))
    plt.title(f"Variance of {word} data")
    plt.grid(True, axis='x', linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.subplots_adjust(top=0.9)
    fig_var.legend(handles=legend, loc='upper center', bbox_to_anchor=(0.5, 0.99
    plt.show()

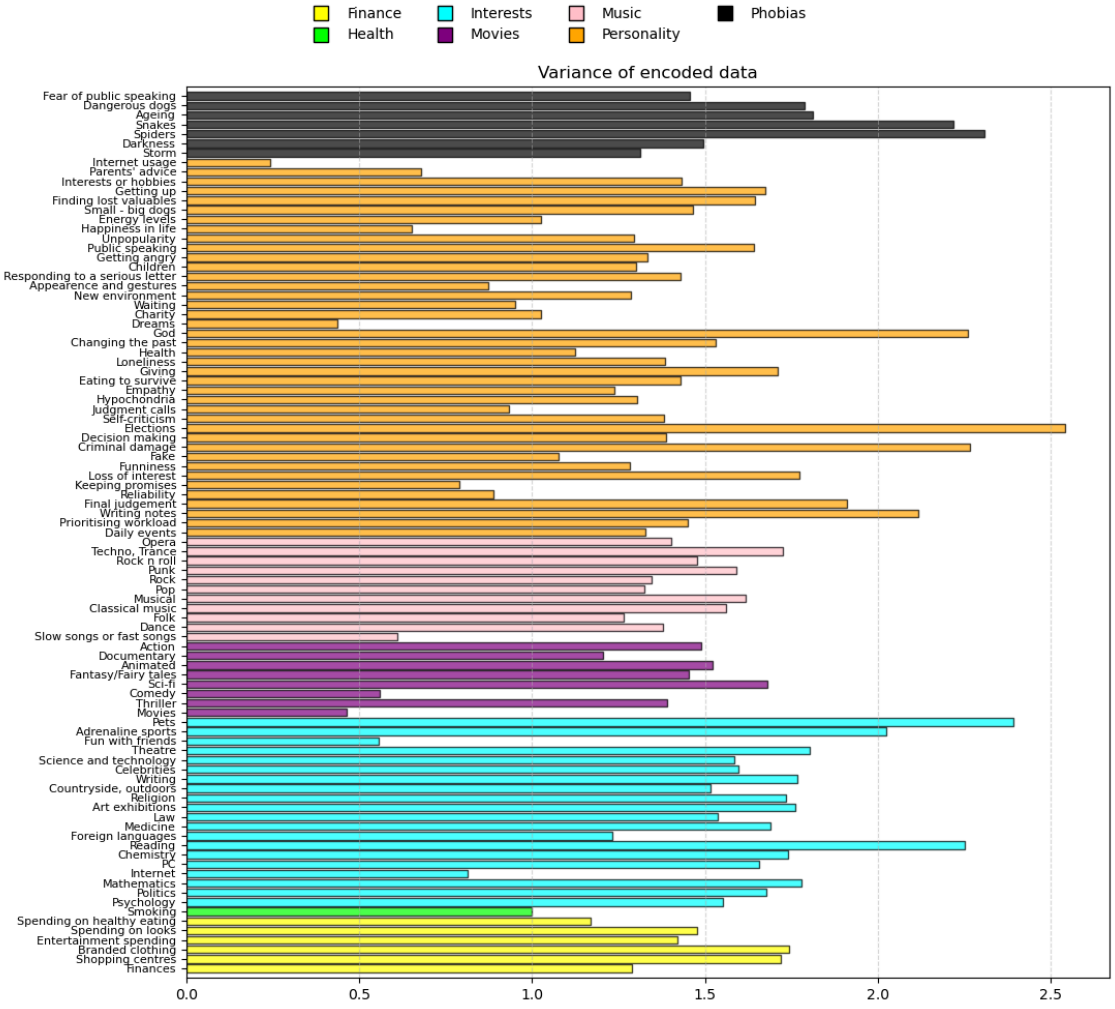
enc_variance = np.var(responses_ft_enc, axis=0)
pp_variance = np.var(responses_ft_pp, axis=0)

feature_names = responses_ft.columns.values
y_pos = np.arange(len(feature_names))

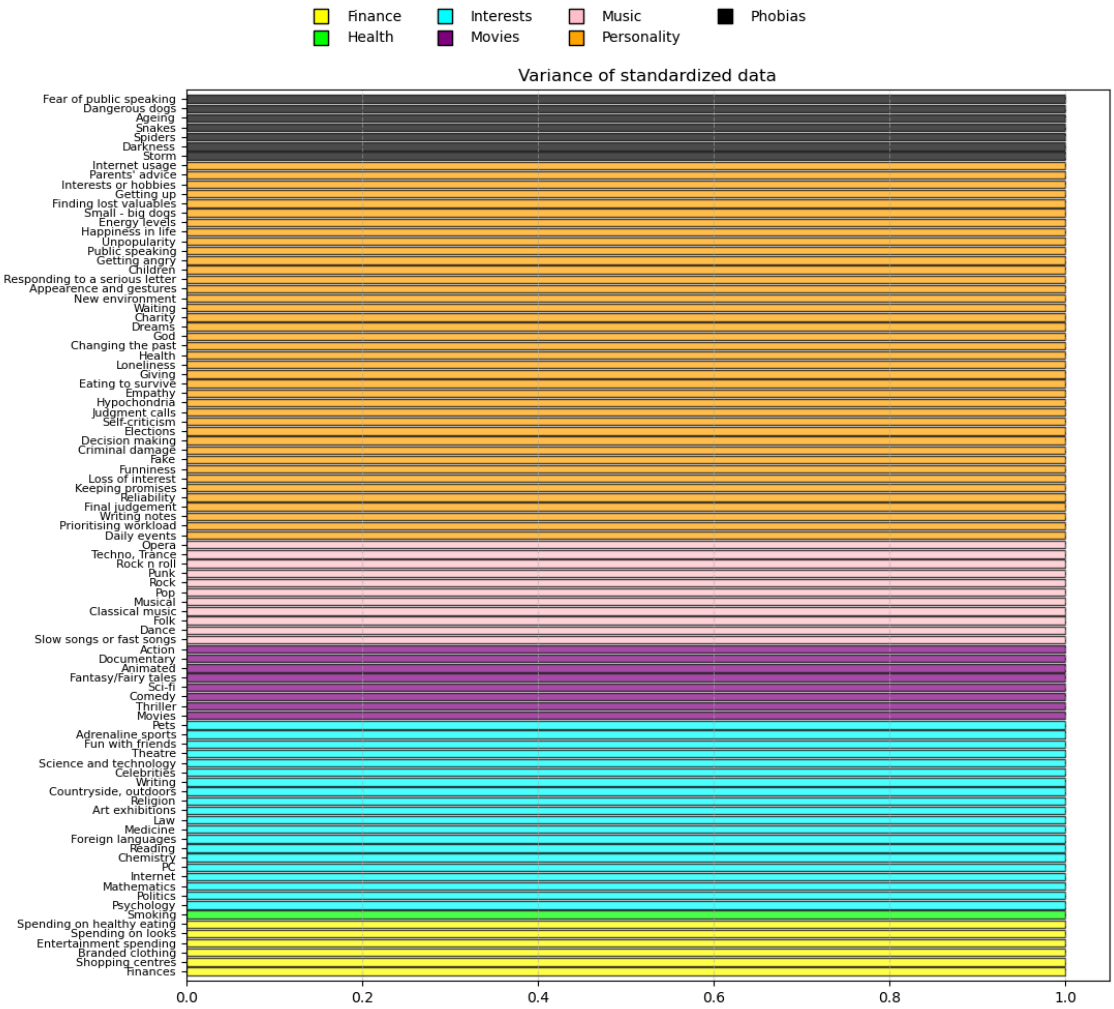
categories_colors, categories_legend = create_colors_and_legend(feature_names, f

plt.close("all")
variance_graph(enc_variance, y_pos, feature_names, "encoded", categories_colors,
variance_graph(pp_variance, y_pos, feature_names, "standardized", categories_col
```

Figure


Finance
Health
Interests
Movies
Music
Personality
Phobias
Variance of encoded data
Fear of public speaking
Dangerous dogs
Ageing
Snakes
Spiders
Darkness
Storm
Internet usage
Parents' advice
Interests or hobbies
Getting up
Finding lost valuables
Small - big dogs
Energy levels
Happiness in life
Unpopularity
Public speaking
Getting angry
Children
Responding to a serious letter
Appearence and gestures
New environment
Waiting
Charity
Dreams
God
Changing the past
Health
Loneliness
Giving
Eating to survive
Empathy
Hypochondria
Judgment calls
Self-criticism
Elections
Decision making
Criminal damage
Fake
Funniness
Loss of interest
Keeping promises
Reliability
Final judgement
Writing notes
Prioritising workload
Daily events
Opera
Techno, Trance
Rock n roll
Punk
Rock
Pop
Musical
Classical music
Folk
Dance
Slow songs or fast songs
Action
Documentary
Animated
Fantasy/Fairy tales
Sci-fi
Comedy
Thriller
Movies
Pets
Adrenaline sports
Fun with friends
Theatre
Science and technology
Celebrities
Writing
Countryside, outdoors
Religion
Art exhibitions
Law
Medicine
Foreign languages
Reading
Chemistry
PC
Internet
Mathematics
Politics
Psychology
Smoking
Spending on healthy eating
Spending on looks
Entertainment spending
Branded clothing
Shopping centres
finances
0.0
0.5
1.0
1.5
2.0
2.5

Figure



Comment the results obtained for the variances (max 150 words):

If we applied PCA to the non standardized data, the algorithm would mistakenly identify these large-scale features as the most 'informative' components simply due to their magnitude. In contrast, the standardized dataset shows a uniform variance of 1 for all features. This confirms that the scaling was successful. By equalizing the variance, we ensure that PCA will now extract components based on the true correlation structure of the data, rather than being biased by the arbitrary units of measurement

Write the code for computing all the n PCs of the two datasets, separately, and for visualizing the curves of cumulative explained variances:

In [10]:
```
def cumulative_variance_graph(pca_passed, word):
    plt.figure(figsize=(11, 7))
    plt.bar(np.arange(pca_passed.n_features_in_), pca_passed.explained_variance_
    plt.bar(np.arange(pca_passed.n_features_in_), np.insert(np.cumsum(pca_passed
    plt.plot(np.cumsum(pca_passed.explained_variance_ratio_), 'r')
    plt.title(f'{word} features')
    plt.xticks(ticks=np.arange(pca_passed.n_features_in_), labels=[i for i in ra
    plt.xlim(-1, pca_passed.n_features_in_)
    plt.xlabel('Principal components')
    plt.ylabel('Cumulative explained variance (%)')
    plt.grid(color="black", axis="y")
```

```
    plt.tight_layout()
    plt.show()

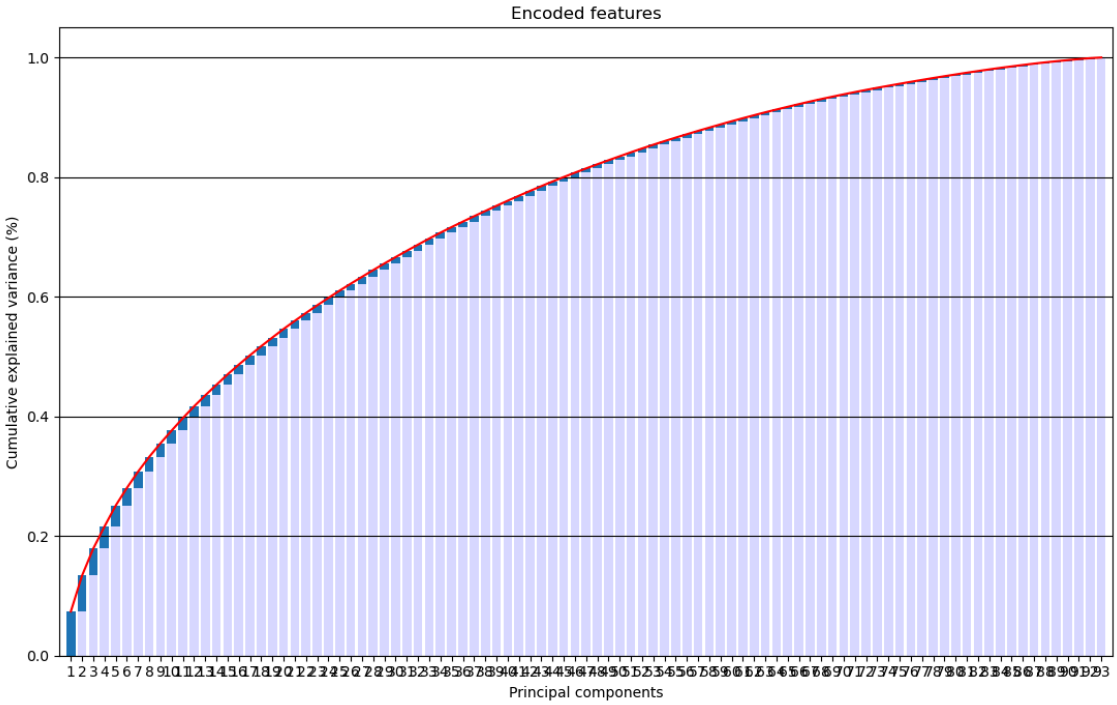
plt.close("all")

pca_enc = PCA()
pca_pp = PCA()

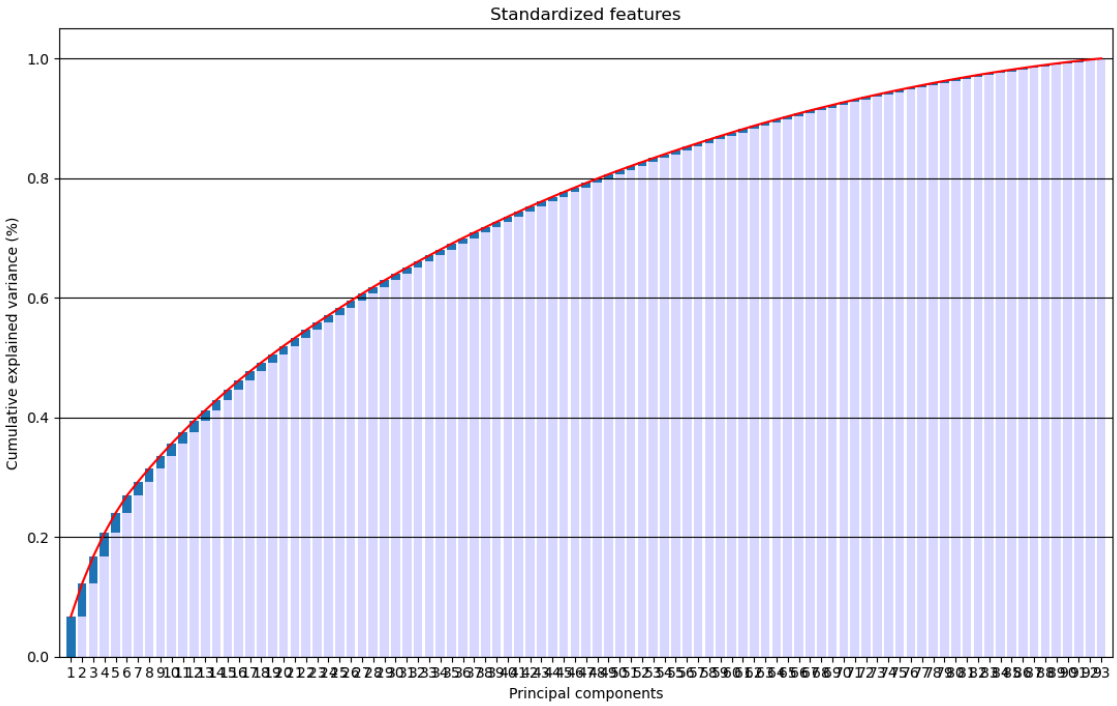
pca_enc.fit(responses_ft_enc)
pca_pp.fit(responses_ft_pp)

cumulative_variance_graph(pca_enc, "Encoded")
cumulative_variance_graph(pca_pp, "Standardized")
```

Figure



Figure



Comment the results obtained for the cumulative explained variances, knowing the vaues in the datasets and the fetures' variances (max 150 words):

The unscaled dataset exhibits a steeper ascent, this occurs because variables with large numerical magnitudes dominate the variance calculation. In contrast, the standardized dataset displays a much more gradual slope. It requires significantly more components to explain a high percentage of variance.

Exercise 3. Dimensionality Reduction and PC Interpretation

In this exercise, you have to do the following operations:

1. For the dataset *responses_ft_pp*, compute a new PCA for performing a dimensionality reduction with respect to m dimensions. The value of m must be

$$m = \min\{m', 5\}\,,$$

   where m' is the value required for obtaining 33% of the total variance.
2. Visualize as a barplot the explained variance (as percentage) for each PC, and report the preserved explained variance (as percentage) by the m PCs.
3. Visualize all the PCs as barplots and give an interpretation and a name to them, **motivating your choices**.
4. Transform the *responses_ft_pp* data into their m-dimensional representation via PCA. Store the transformed data in the variable *responses_ft_pca*;
5. Visualize the the score graph. If $m > 3$, plot the score graph with respect to the first 3 PCs. All the **plots must show the names of the PCs (given at the previous step) on the axes** for better understanding the results.

Write the code for computing the new PCA, for visualizing the m PCs as barplots:

In [13]:
```
plt.close("all")
m_prime = np.argmax(np.cumsum(pca_pp.explained_variance_ratio_) > 0.33) + 1
m = min(5, m_prime)

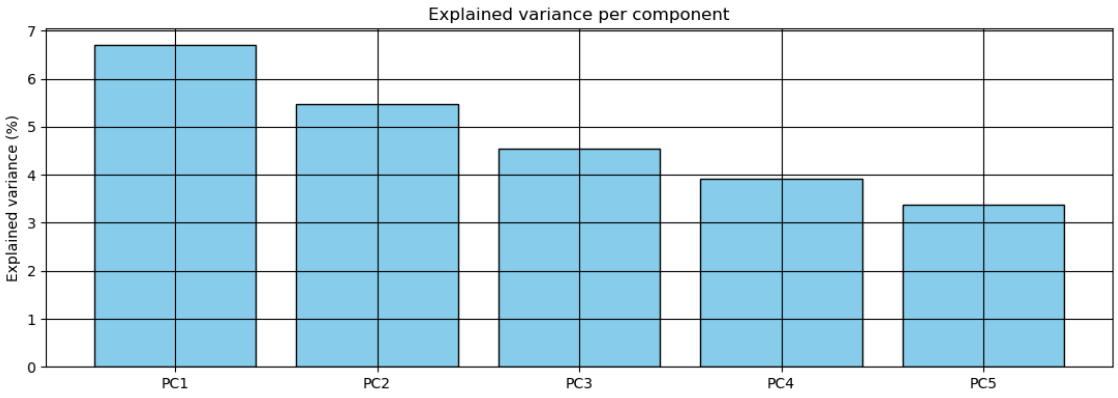
pca_ft = PCA(n_components=m)
pca_ft.fit(responses_ft_pp)

exp_var_ratio = pca_ft.explained_variance_ratio_
exp_var_percentage = exp_var_ratio * 100
total_preserved_variance = np.sum(exp_var_percentage)

print('')
print(f"Total variance preserved from the first {m} PC: {total_preserved_varianc
plt.figure(figsize=(11,4))
bars = plt.bar([f'PC{i+1}' for i in range(m)], exp_var_percentage, color='skyblu
plt.title(f'Explained variance per component')
plt.ylabel('Explained variance (%)')
plt.grid(True, color="black")
plt.tight_layout()
plt.show()
```

```
Total variance preserved from the first 5 PC: 24.03%
```

Figure



In [14]:
```
plt.close("all")

PCs = pca_ft.components_

n_responses = len(responses_ft)
n_features = len(feature_names)

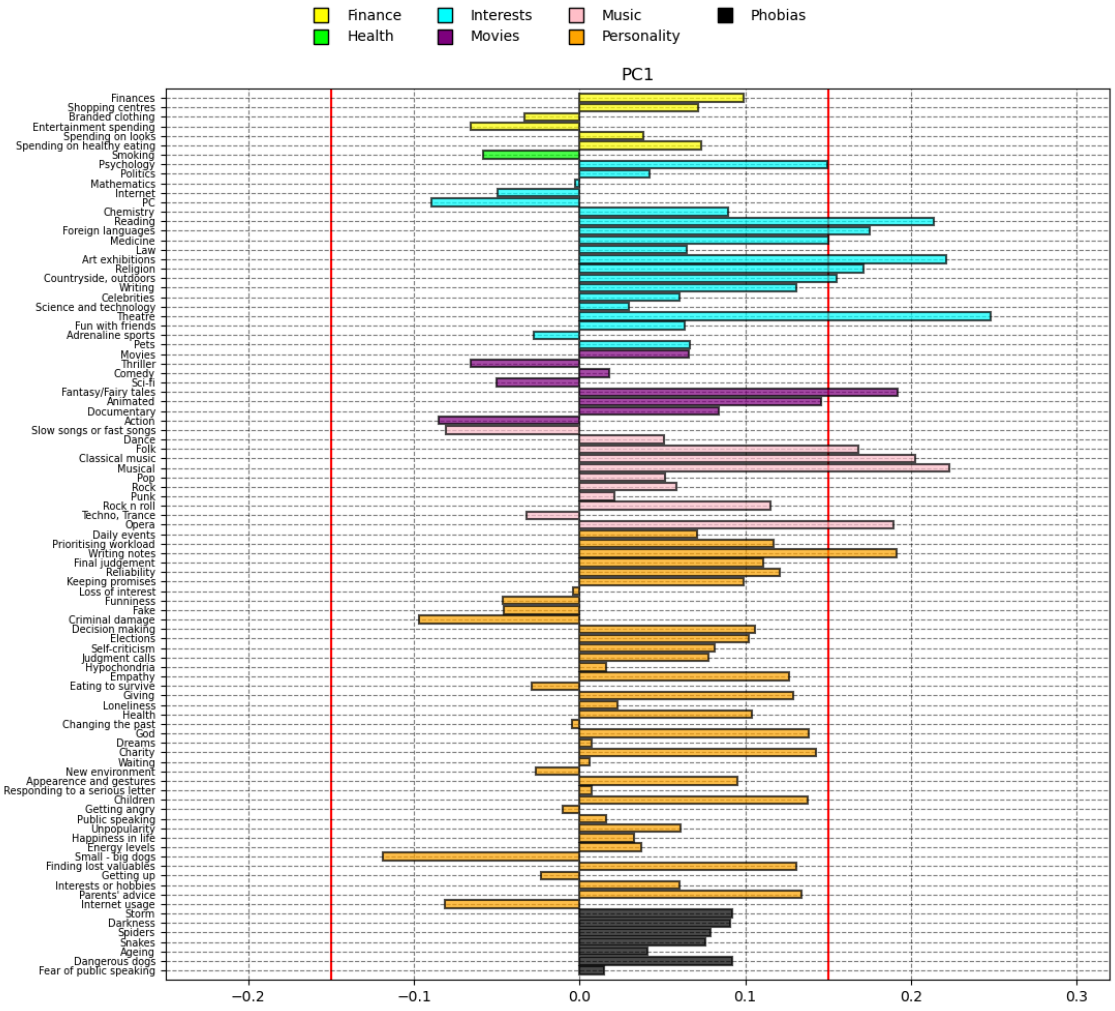
feature_indices = range(n_features)

for i in range(m):
    fig_pca = plt.figure(figsize=(11, 10))
    ymin_line = min(feature_indices) - 5
    ymax_line = max(feature_indices) + 5
    plt.ylim(ymin_line, ymax_line)
    plt.vlines(0.15, ymin_line, ymax_line, color="red", zorder=1)
    plt.vlines(-0.15, ymin_line, ymax_line, color="red", zorder=1)
    plt.barh(feature_indices, PCs[i], color=categories_colors, alpha=0.7, height
    plt.title(f'PC{i+1}')
    plt.grid(True, color="black", linestyle='--', alpha=0.5)
```

```
    plt.xlim(-0.25, 0.32)
    plt.ylim(-1, n_features)
    plt.yticks(feature_indices, feature_names, fontsize=7)
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.subplots_adjust(top=0.9)
    fig_pca.legend(handles=categories_legend, loc='upper center', bbox_to_anchor
    plt.show()

    high_value_ind = PCs[i] >= 0.15
    low_value_ind = PCs[i] <= -0.15
    high_features = responses_ft_pp.columns[high_value_ind].values
    low_features = responses_ft_pp.columns[low_value_ind].values
    print(f"High value positive components: {high_features}")
    print(f"\nHigh value negative components: {low_features}")
```

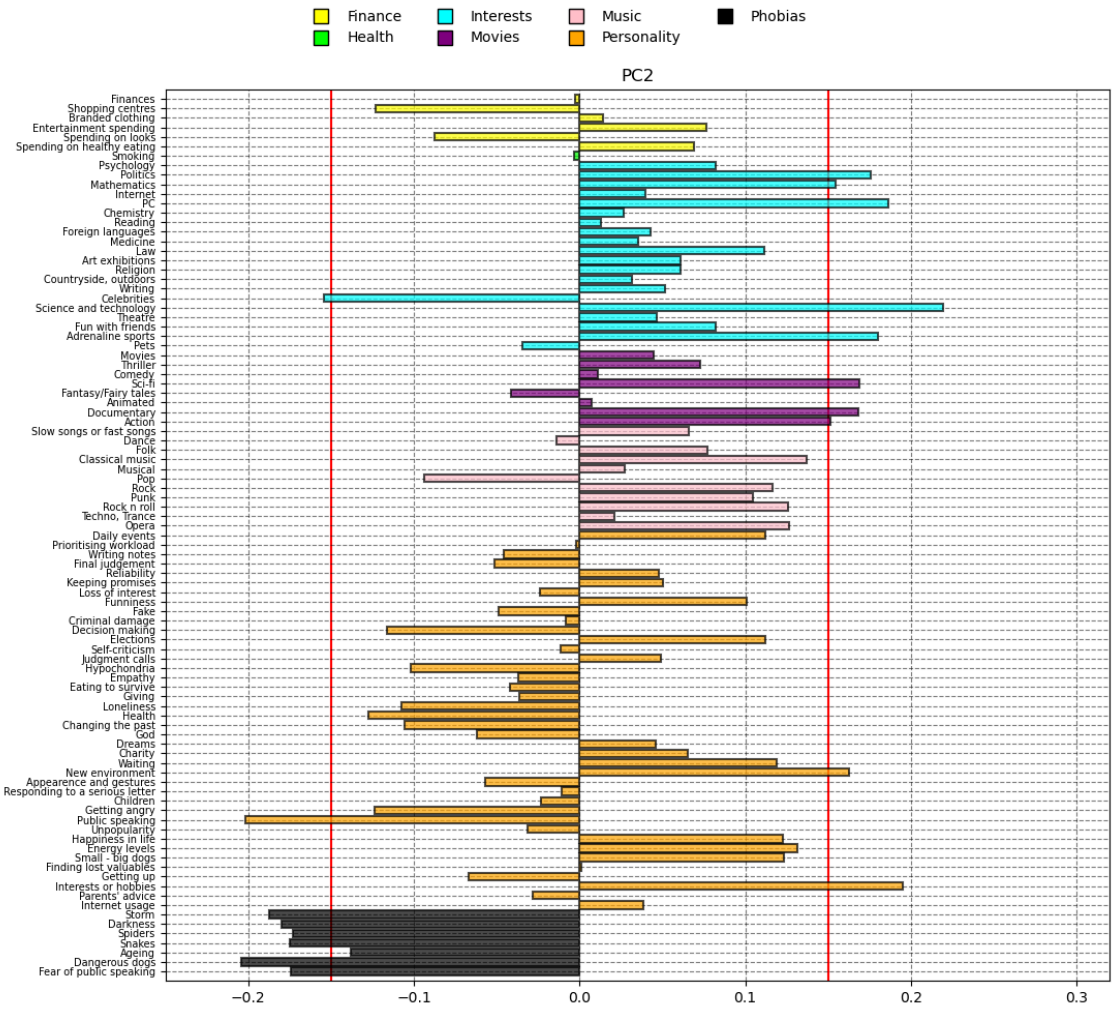
Figure




```
High value positive components: ['Reading' 'Foreign languages' 'Medicine' 'Art ex
hibitions' 'Religion'
 'Countryside, outdoors' 'Theatre' 'Fantasy/Fairy tales' 'Folk'
 'Classical music' 'Musical' 'Opera' 'Writing notes']

High value negative components: []
```

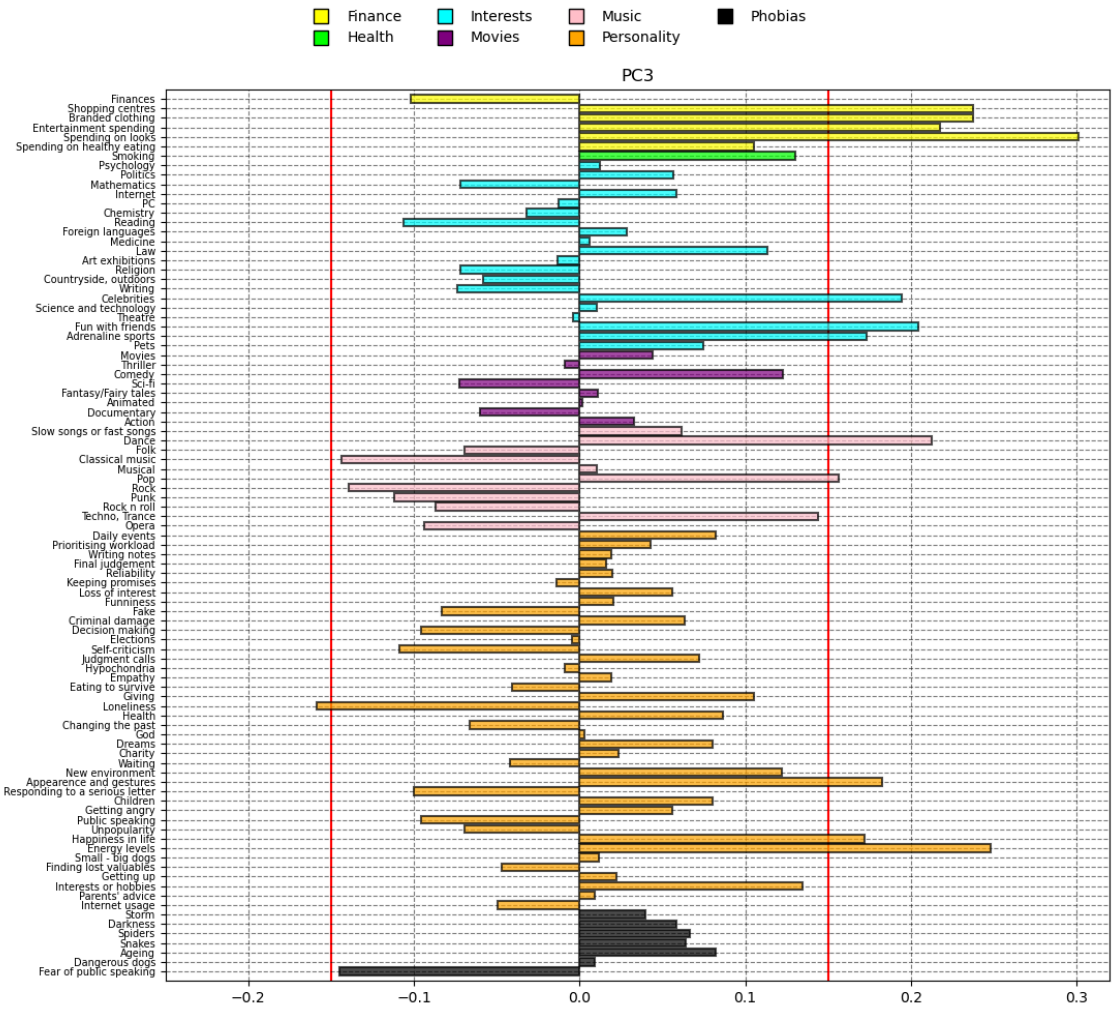
Figure




```
High value positive components: ['Politics' 'Mathematics' 'PC' 'Science and techn
ology'
 'Adrenaline sports' 'Sci-fi' 'Documentary' 'Action' 'New environment'
 'Interests or hobbies']

High value negative components: ['Celebrities' 'Public speaking' 'Storm' 'Darknes
s' 'Spiders' 'Snakes'
 'Dangerous dogs' 'Fear of public speaking']
```

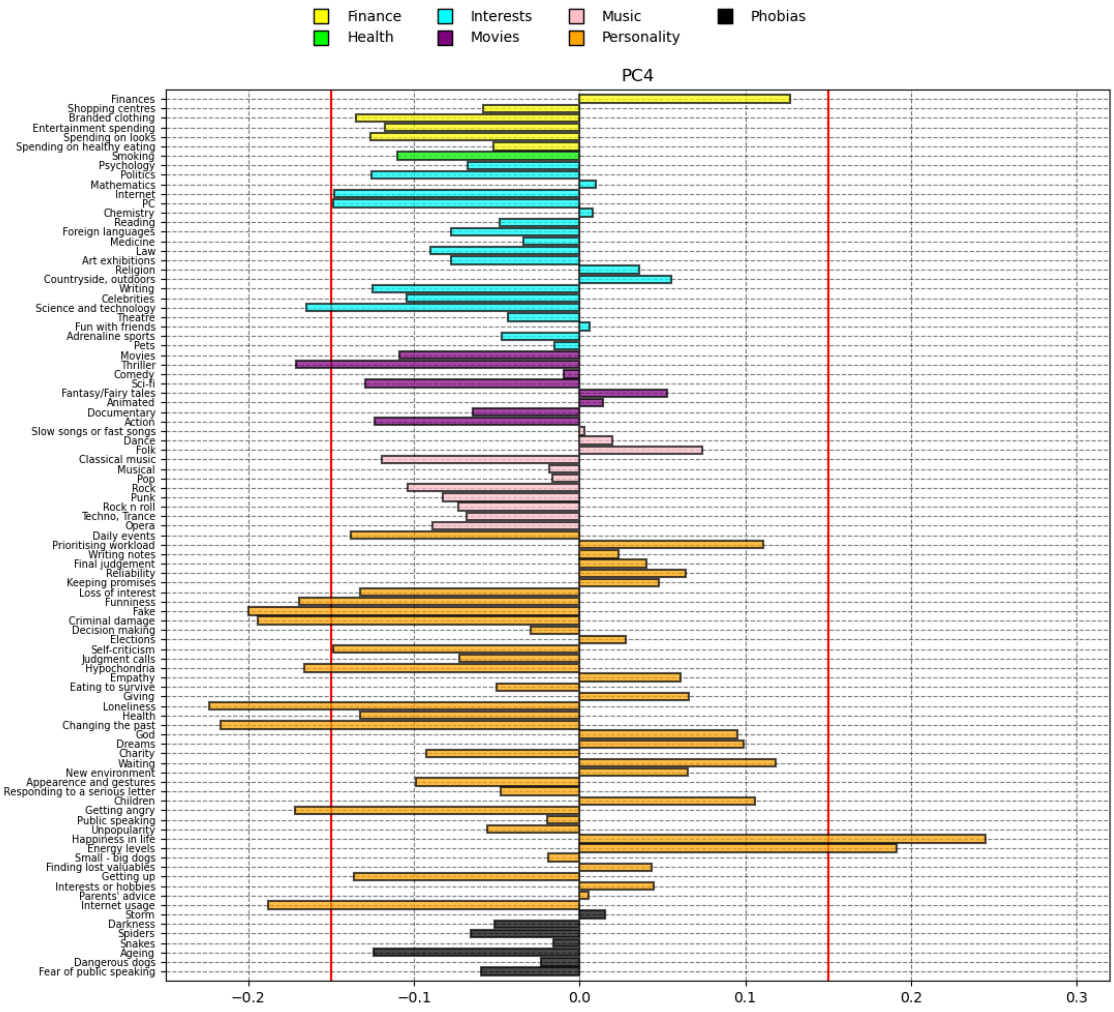
Figure




```
High value positive components: ['Shopping centres' 'Branded clothing' 'Entertain
ment spending'
 'Spending on looks' 'Celebrities' 'Fun with friends' 'Adrenaline sports'
 'Dance' 'Pop' 'Appearence and gestures' 'Happiness in life'
 'Energy levels']

High value negative components: ['Loneliness']
```

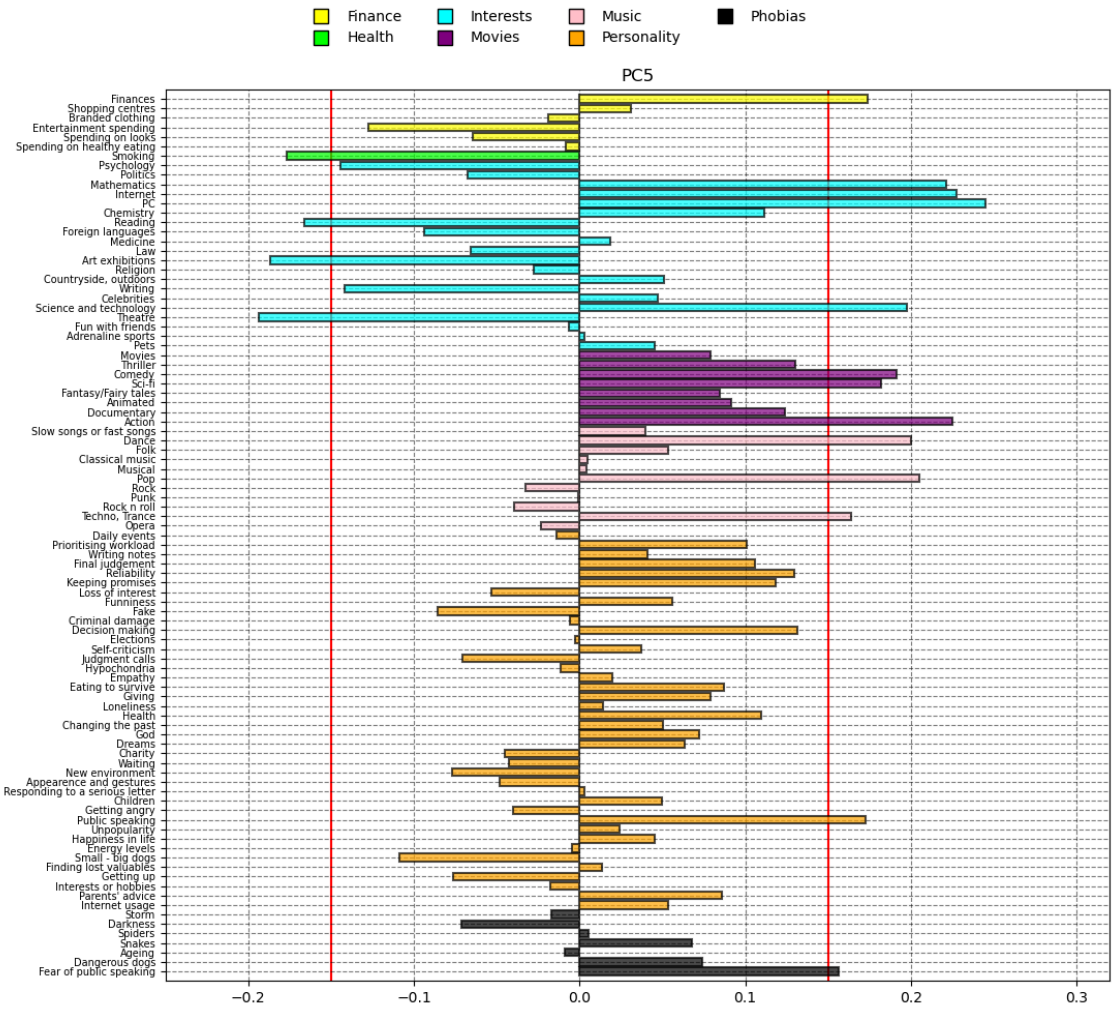
Figure




```
High value positive components: ['Happiness in life' 'Energy levels']

High value negative components: ['Science and technology' 'Thriller' 'Funniness'
'Fake' 'Criminal damage'
 'Hypochondria' 'Loneliness' 'Changing the past' 'Getting angry'
 'Internet usage']
```

Figure




```
High value positive components: ['Finances' 'Mathematics' 'Internet' 'PC' 'Scienc
e and technology'
 'Comedy' 'Sci-fi' 'Action' 'Dance' 'Pop' 'Techno, Trance'
 'Public speaking' 'Fear of public speaking']

High value negative components: ['Smoking' 'Reading' 'Art exhibitions' 'Theatre']
```

For each PC, write the name you assigned to it and a brief interpretation that motivate the choice (max 100 words per PC):

PC1: **Classical Intellectual**

We see that the principal features in PC1 are all in the field of classical arts, such as opera, art, religion and countryside, all features of a classical intellectual who appreciates this fields.

PC2: **Rational Adventurer**

This is suitable because it has high positive values among stem fields, such as PC, science and maths, and in adventures, such as sports, new environment and action. It is also not afraid of the public and not subject to common phobias as snakes, spiders and darkness. Perfect for a stem intelligent person who loves adventure.

PC3: **Popular Trend Icon**

This works because it thrives in shopping, spending, appearance, popularity and

happiness, suggesting that it uses a lot of energy to keep at the highest of level for fashion.

PC4: **Happy Natural**
Thanks to high positive value in happiness and energy and high negative values in science, fake, criminal, getting angry and internet, this name suits perfectly this PC, a down to earth optimist focused on happiness.

PC5: **Digital Pragmatist**
This title is a great fit, revolving around high positive value like math, finance, internet, techno and action, while high negative values such as reading, art, theatre.

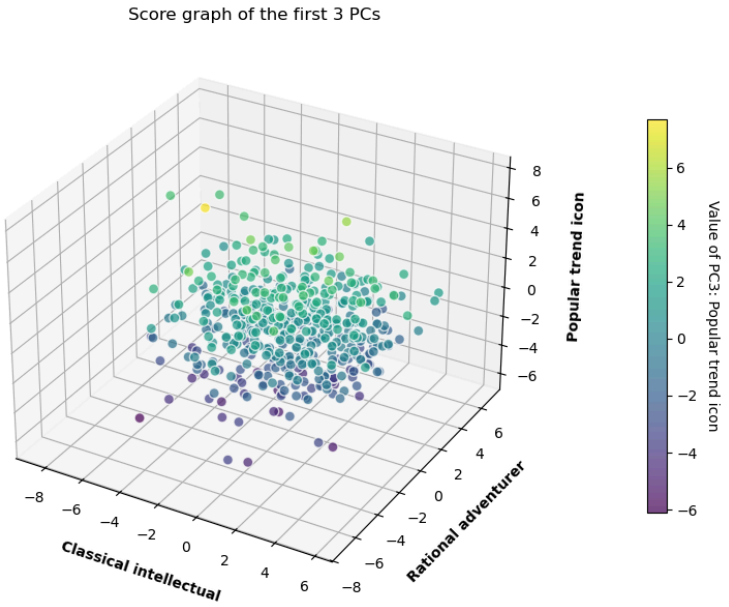
Write the code for visualizing the score graph (with PC names on the axis):

In [18]:
```
plt.close("all")

transformed_data = pca_ft.transform(responses_ft_pp)
PC_names = ["Classical intellectual", "Rational adventurer", "Popular trend icon
responses_ft_pca = pd.DataFrame(transformed_data, columns=PC_names)

fig = plt.figure(figsize=(11, 6))
ax = fig.add_subplot(111, projection='3d')
x_col = PC_names[0]
y_col = PC_names[1]
z_col = PC_names[2]
x_data = responses_ft_pca[x_col]
y_data = responses_ft_pca[y_col]
z_data = responses_ft_pca[z_col]
scatter = ax.scatter(x_data, y_data, z_data, c=z_data, cmap='viridis', s=50, alp
ax.set_xlabel(x_col, fontsize=10, fontweight='bold', labelpad=10)
ax.set_ylabel(y_col, fontsize=10, fontweight='bold', labelpad=10)
ax.set_zlabel(z_col, fontsize=10, fontweight='bold', labelpad=10)
ax.set_title("Score graph of the first 3 PCs", fontsize=12, pad=10)
cbar = plt.colorbar(scatter, ax=ax, shrink=0.7, pad=0.1)
cbar.set_label(f"Value of PC3: {z_col}", rotation=270, labelpad=15)
plt.tight_layout()
plt.show()
```

Figure



Exercise 4. k-Means

In this exercise, you have to do the following operations:

1. Run the k-Means for clustering the data of *responses_ft_pca*, **setting the input argument *random_state* equal to the variable *random_seed*** (i.e., the minimum of the Student IDs).

   In particular, **use the silohuette score for identifying the best value for** $k \in \{3, \ldots, 10\}$ and show it by plotting how the score changes w.r.t. k.

2. Plot the score graph again, but add the centroids of the cluster and color the points according to their cluster.

3. Visualize the centroids coordinates as barplots and **give a name and an interpretation to them by exploiting the PC names**.

Write the code for performing the items of the list above:

In [21]:
```
plt.close("all")

k_values = [3, 4, 5, 6, 7, 8, 9, 10]
scores = []
best_score = -np.inf

for k in k_values:
    kmeans_k = KMeans(n_clusters=k, random_state=random_seed)
    labels_k = kmeans_k.fit_predict(responses_ft_pca)
    score_k = silhouette_score(responses_ft_pca, labels_k)

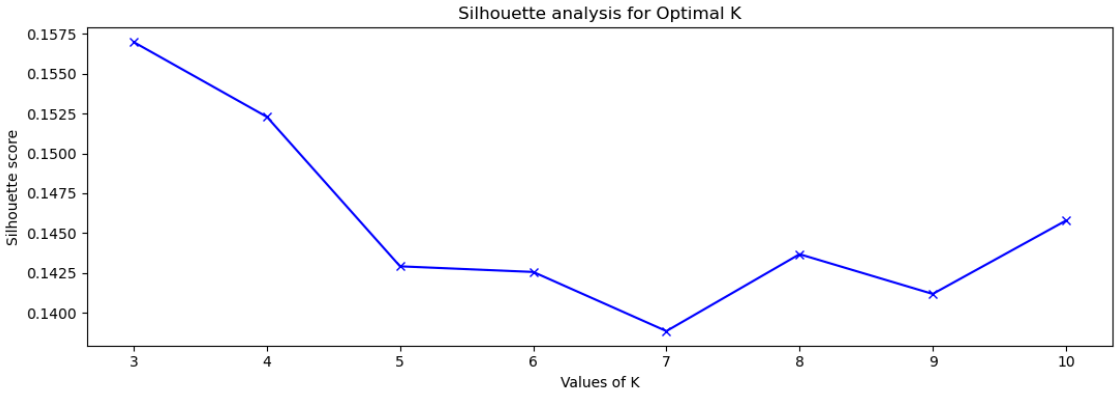
    scores.append(score_k)
    print(f"n_clusters = {k:<2} -> silhouette score = {score_k:.3f}")

plt.figure(figsize=(11,4))
plt.plot(k_values, scores, 'bx-')
```

```
plt.xlabel('Values of K')
plt.ylabel('Silhouette score')
plt.title('Silhouette analysis for Optimal K')
plt.tight_layout()
plt.show()
```

```
n_clusters = 3  -> silhouette score = 0.157
n_clusters = 4  -> silhouette score = 0.152
n_clusters = 5  -> silhouette score = 0.143
n_clusters = 6  -> silhouette score = 0.143
n_clusters = 7  -> silhouette score = 0.139
n_clusters = 8  -> silhouette score = 0.144
n_clusters = 9  -> silhouette score = 0.141
n_clusters = 10 -> silhouette score = 0.146
```

Figure



In [23]:

```
def score_graph(pc_names, pred, num_cluster, centr, pca_features=None):
    fig = plt.figure(figsize=(11, 6))
    ax = fig.add_subplot(111, projection='3d')
    if pca_features is not None:
        ax.scatter(pca_features[pc_names[0]], pca_features[pc_names[1]], pca_fea
    ax.scatter(centr[0, :3], centr[1, :3], centr[2, :3], c='black', s=200, edgec
    ax.set_xlim(-8, 8)
    ax.set_ylim(-8, 8)
    ax.set_zlim(-8, 8)
    ax.set_xlabel(pc_names[0])
    ax.set_ylabel(pc_names[1])
    ax.set_zlabel(pc_names[2])
    ax.set_title(f'Score Plot: {num_cluster} Clusters on First 3 PCs')
    plt.tight_layout()
    plt.show()

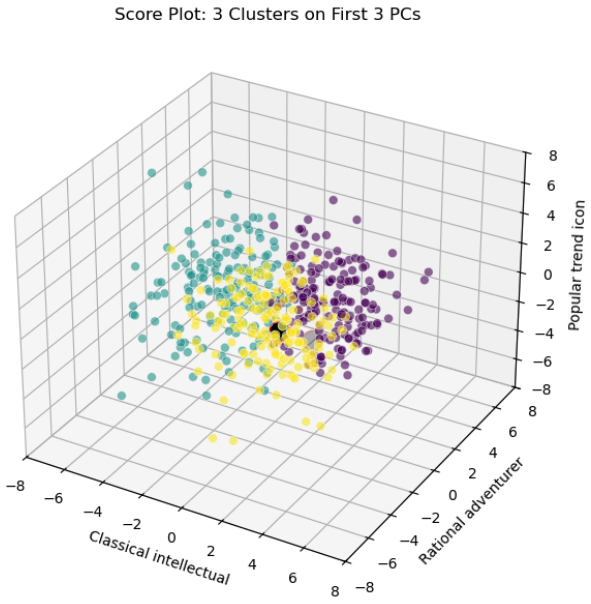
plt.close("all")

best_k = k_values[np.argmax(scores)]

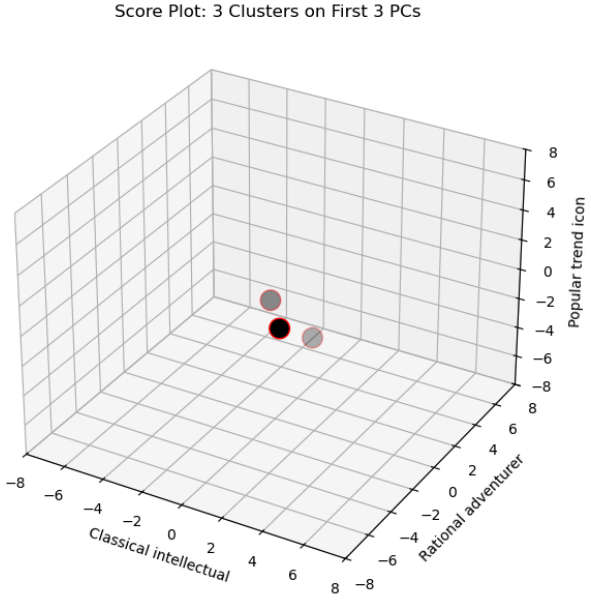
kmeans = KMeans(n_clusters=best_k, random_state=random_seed)
predictions = kmeans.fit_predict(responses_ft_pca)
centroids = kmeans.cluster_centers_

score_graph(PC_names, predictions, best_k, centroids, responses_ft_pca)
score_graph(PC_names, predictions, best_k, centroids)
```

Figure



Figure



In [26]:

```
plt.close("all")

print(pd.DataFrame(centroids, columns=PC_names, index=[f"Centroid {k+1}" for k i

maxs_features_pca = responses_ft_pca.max(axis=0)
mins_features_pca = responses_ft_pca.min(axis=0)

indices_colors_centroids = np.linspace(0, 1, m)
colors_centroids = [plt.cm.viridis(i) for i in indices_colors_centroids]
legend_centroids = []

for i in range(m):
    legend_centroids.append(Line2D([0], [0], marker='o', color='w', label=PC_nam

fig, ax = plt.subplots(1, best_k, figsize=(11, 4))
for i in range(0, best_k):
    ax[i].bar(np.arange(centroids.shape[1]), maxs_features_pca, color=colors_cen
    ax[i].bar(np.arange(centroids.shape[1]), mins_features_pca, color=colors_cen
```

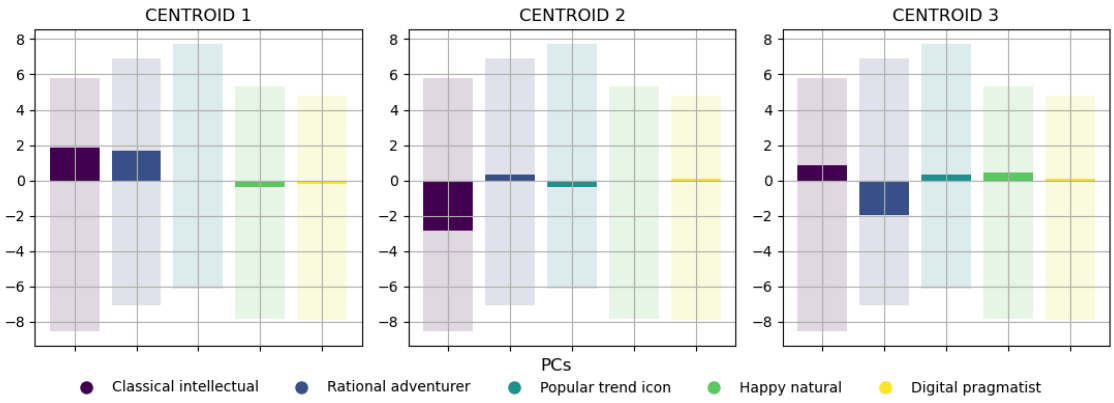
```
    ax[i].bar(np.arange(centroids.shape[1]), centroids[i, :], color=colors_centr
    ax[i].grid(visible=True, which='both')
    ax[i].set_title(f'CENTROID {i+1}')
    ax[i].set_xticklabels(labels=[])

fig.legend(handles=legend_centroids, loc='lower center', bbox_to_anchor=(0.5, 0)
plt.tight_layout()
plt.subplots_adjust(bottom=0.15)
plt.show()
```

	Classical intellectual	Rational adventurer	Popular trend icon	\
Centroid 1	1.846038	1.683063	-0.033824	
Centroid 2	-2.825871	0.356171	-0.344750	
Centroid 3	0.875505	-1.981790	0.359857	

	Happy natural	Digital pragmatist
Centroid 1	-0.384380	-0.191944
Centroid 2	-0.089617	0.116608
Centroid 3	0.460448	0.076964

Figure



For each Centroid, write the name you assigned to it and a brief interpretation that motivate the choice by exploiting the PC names(max 100 words per centroid):

Centroid 1: "The Classical Adventurer"

High PC1 1.84 and High PC2 1.68.

This group is unique because they score highly in both Classical Intellectual and Rational Adventurer. They are well-rounded, academic, and adventurous.

Centroid 2: "The Anti-Academic"

Extremely Low PC1 -2.82.

This value is massively negative. Since PC1 represents Classical Intellectual, this group strongly dislikes those traditional culture topics.

They define themselves by rejecting the classical intellectual lifestyle, preferring more modern or practical entertainments over traditional hobbies.

Centroid 3: "The Cautious Humanist"

Very Low PC2 -1.98 and Moderate PC1 0.88.

A negative score in PC2 suggests this group might have more phobias and, in general,

might be more afraid than normal, also dislikes STEM and Action.
But the high score in PC1 suggests that they still enjoy the arts, reading and the more classical culture.

Exercise 5. Cluster External Evaluations

In this exercise, you have to do the following operations:

1. Select a subset meaningful labels for performing an external evaluation of the clustering results.
2. For each selected label, visualize the distribution of the label in each cluster and in the whole dataset.
3. Visualize the score graph with dots colored with respect to the label value; then, visualize the clusters in separated score-graphs, coloring the points according to the label values.

List the Labels you consider meaningful for an external cluster evaluation and motivate your choice (max 50 words per label):

For the external evaluation we will be considering 4 labels:

1. Gender
   We chose gender to see if in this dataset there is some correlation between male-adventurers, female-cautious humanist and all the other possibilities.
2. Education
   Analysing education allows us to see whether the most educated people are the one more interested in classical arts/STEM rather than in adventure for example.
3. Only child
   It is interesting to see if effectively exists a correlation between different archetypes and having siblings, does being an only child lead you to a certain personality?
4. Age
   Does age define what people are most interested in? For example, will younger people be more adventurous/anti-academic?
5. Hometown type
   We selected the hometown type to see if different growing conditions affect someone's personality, for example if city means more interested in tech and if village means leaning toward nature.

Write the code for the visualizations cited in item 2 above:

In [ ]:
```
plt.close("all")

centroids_names = ["Classical adventurer", "Anti-academic", "Cautious humanist"]

PC_important_names = ["Classical intellectual", "Rational adventurer", "Popular
PC_important_index = [0, 1, 3]
important_labels = ["Gender", "Education", "Only child", "Age", "Home Town type"
```

```
color_gender = responses_lb['Gender'].map({'male': '#1f77b4', 'female': '#e377c2
colors_inferno = ['#fcffa4', '#fab62d', '#ed6925', '#b52f8c', '#6b116f', '#1b0c4
education_map = { 'currently a primary school pupil': '#fcffa4', 'primary school
color_education = responses_lb['Education'].map(education_map)
color_only_child = responses_lb['Only child'].map({'yes': '#FFD700', 'no': '#000
norm_age = plt.Normalize(responses_lb['Age'].min(), responses_lb['Age'].max())
cmap_age = plt.get_cmap('viridis_r')
colors_age = cmap_age(norm_age(responses_lb['Age']))
color_home_town = responses_lb['Home Town Type'].map({'city': '#7f7f7f', 'villag
all_colors = [color_gender, color_education, color_only_child, colors_age, color

gender_legend = [Line2D([0], [0], marker='o', color='w', label='Male', markerfac
                 Line2D([0], [0], marker='o', color='w', label='Female', markerf
education_legend = [Line2D([0], [0], marker='o', color='w', label='doctorate deg
                    Line2D([0], [0], marker='o', color='w', label='masters degre
                    Line2D([0], [0], marker='o', color='w', label='college/bache
                    Line2D([0], [0], marker='o', color='w', label='secondary sch
                    Line2D([0], [0], marker='o', color='w', label='primary schoo
                    Line2D([0], [0], marker='o', color='w', label='currently a p
only_child_legend = [Line2D([0], [0], marker='o', color='w', label='Yes', marker
                     Line2D([0], [0], marker='o', color='w', label='No', markerf
home_town_legend = [Line2D([0], [0], marker='o', color='w', label='City', marker
                    Line2D([0], [0], marker='o', color='w', label='Village', mar
all_legends = [gender_legend, education_legend, only_child_legend, "", home_town

for i in range(len(important_labels)):
    fig = plt.figure(figsize=(11, 5))
    fig.suptitle(f'{important_labels[i]} distribution')
    ax = fig.add_subplot(1, 4, 1, projection='3d')
    scatter = ax.scatter(responses_ft_pca[PC_important_names[0]], responses_ft_p
    ax.set_xlabel(PC_important_names[0])
    ax.set_ylabel(PC_important_names[1])
    ax.set_zlabel(PC_important_names[2])
    ax.set_title(f'Whole database')

    for j in range(best_k):
        mask = predictions == j
        temp_ft = responses_ft_pca[mask]

        ax = fig.add_subplot(1, 4, j+2, projection='3d')
        scatter = ax.scatter(
            temp_ft[PC_important_names[0]],
            temp_ft[PC_important_names[1]],
            temp_ft[PC_important_names[2]],
            c=all_colors[i][mask],
            s=40,
            alpha=0.6,
            edgecolors='w',
            linewidth=0.5
        )

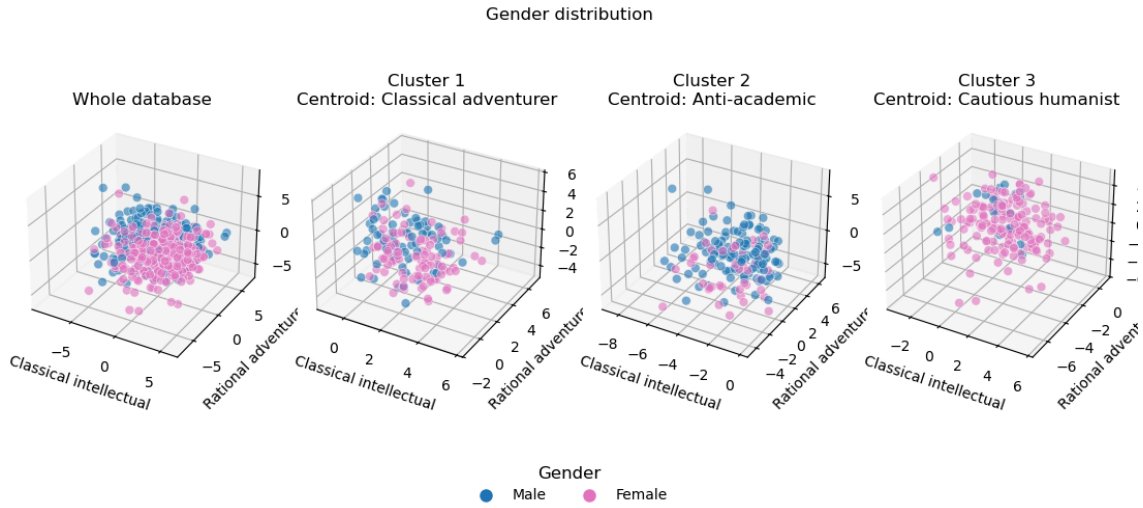
        ax.set_xlabel(PC_important_names[0])
        ax.set_ylabel(PC_important_names[1])
        ax.set_zlabel(PC_important_names[2])
        ax.set_title(f"Cluster {j+1}\nCentroid: {centroids_names[j]}")

    if i == 3:
        norm_age = plt.Normalize(vmin=15, vmax=30)
        sm = plt.cm.ScalarMappable(cmap=cmap_age, norm=norm_age)
        sm.set_array([])
```

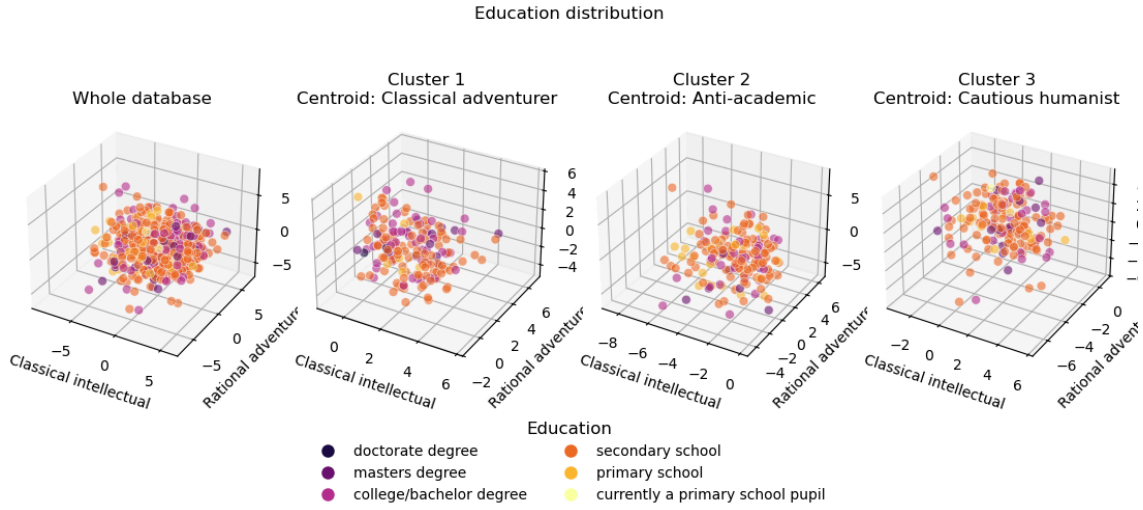
```
        plt.subplots_adjust(left=0.01, right=0.99, top=0.90, bottom=0.25, wspace
        cbar_ax = fig.add_axes([0.15, 0.10, 0.7, 0.04])
        cbar = fig.colorbar(sm, cax=cbar_ax, orientation='horizontal', ticks=ran
        cbar_ax.set_title(f'{important_labels[i]}', fontsize=12, pad=10)
    else:
        fig.legend(handles=all_legends[i], loc='lower center', bbox_to_anchor=(0
        plt.subplots_adjust(left=0.01, right=0.99, top=0.90, bottom=0.15, wspace

    plt.show()
```

Figure

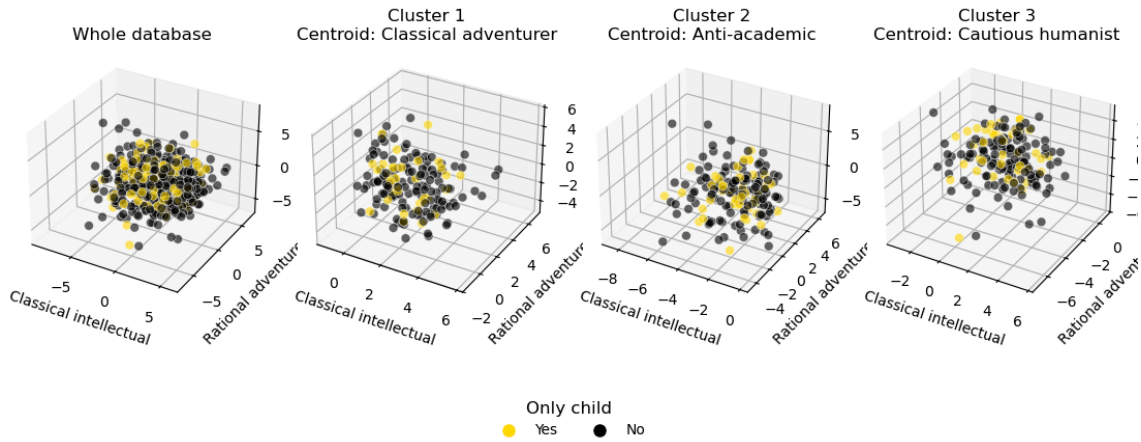


Figure



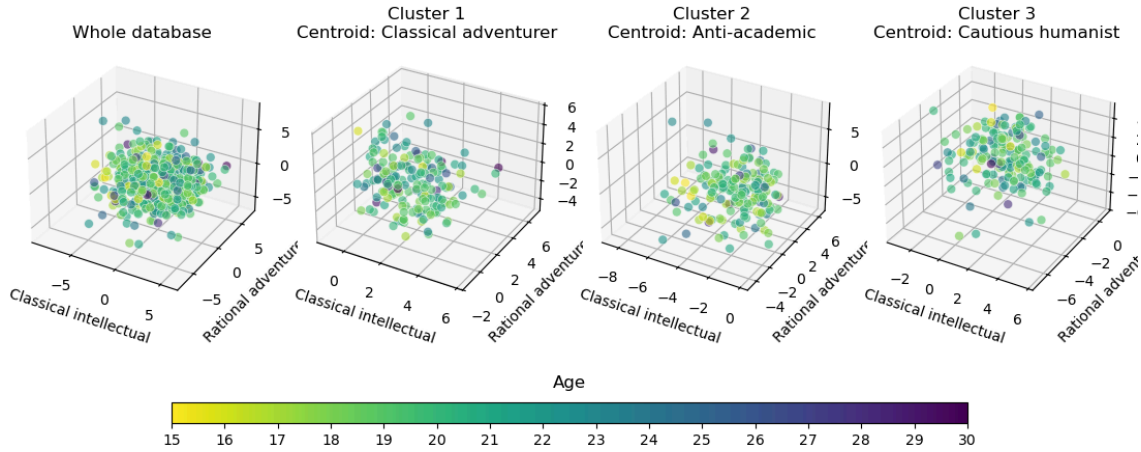
Figure

Only child distribution



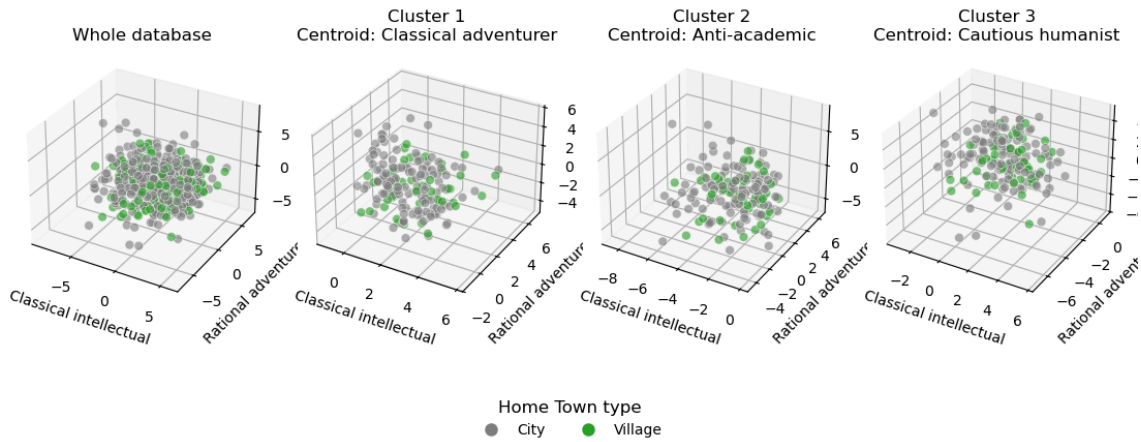
Figure

Age distribution



Figure

Home Town type distribution



For each selected label, comment the results observed in the visualizations (max 100 words per label):

1. Gender

   The 'Gender' label is the only one that is able to tell us something about the dataset, we see that:

- Whole dataset: the males tend to be more 'Rational adventurer' and less 'Classical intellectual', the females instead are the opposite, they lean towards the 'Classical intellectual' rather than 'Rational adventurer'.
- Cluster 1. 'Classical adventurer': in this cluster we see that the distribution between males and females is more or less the same, nothing stands out.
- Cluster 2. 'Anti-academic': this cluster is charachterized by almost exclusively males, this is because males tend to be worse student, therefore making them 'Anti-academic'.
- Cluster 3. 'Cautious humanist': here instead we can see a female dominated cluster, since most of them move away from 'Rational adventurer' and lean towards 'Classical intellectual'. This suggests that they are more inclined to fear and studies, hence the 'Cautous humanist'.

We can observe from the graphs instead that for all the other labels the score graphs doesn't tell us anything, the distribution seems equals and the labels does not lean towards an archetype or another, we can conclude that from the features selected only 'Gender' is able to tell us something.

Exercise 6. Cluster Internal Evaluations

In this exercise, you have to do the following operations:

1. For each cluster, measure the corresponding average silhouette score
2. Visualize the silhouette of the clusters and the general one of the clustering and compare them

Write the code for computing the silhouette scores and for visualizing them:

In [37]:
```
plt.close("all")

sil_avg = silhouette_score(responses_ft_pca, predictions)
sample_sil_values = silhouette_samples(responses_ft_pca, predictions)

fig, ax1 = plt.subplots(1, 1)
fig.set_size_inches(11, 5)
ax1.set_xlim([-0.1, 0.5])
ax1.set_ylim([0, len(responses_ft_pca) + (best_k + 1) * 10])
y_lower = 10
for i in range(best_k):
    ith_cluster_silhouette_values = sample_sil_values[predictions == i]
    ith_cluster_silhouette_values.sort()
    size_cluster_i = ith_cluster_silhouette_values.shape[0]
    y_upper = y_lower + size_cluster_i
    color = cm.nipy_spectral(float(i) / best_k)
    ax1.fill_betweenx(np.arange(y_lower, y_upper), 0, ith_cluster_silhouette_val
    ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i+1))
    y_lower = y_upper + 10

ax1.set_title("Silhouette plot for the various clusters.")
ax1.set_xlabel("Silhouette coefficient values")
ax1.set_ylabel("Cluster labels")
```

```
ax1.axvline(x=sil_avg, color="red", linestyle="--")
ax1.set_yticks([])
ax1.set_xticks([-0.1, 0, 0.1, 0.2, 0.3, 0.4, 0.5])
plt.tight_layout()
plt.show()
```

Figure



In [40]:
```
plt.close("all")

print(f'Silhouette scores')
print(f'Whole dataset: {sil_avg:.4f}')
print(f'{'Cluster 1: ':<15}{np.mean(sample_sil_values[predictions == 0]):.4f}')
print(f'{'Cluster 2: ':<15}{np.mean(sample_sil_values[predictions == 1]):.4f}')
print(f'{'Cluster 3: ':<15}{np.mean(sample_sil_values[predictions == 2]):.4f}')

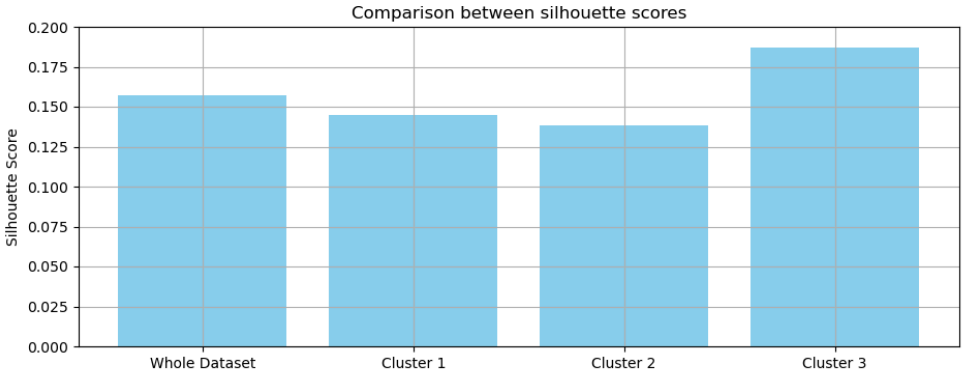
sil_values = [sil_avg]
name_bars = ['Whole Dataset']

for i in range(best_k):
    sil_values.append(np.mean(sample_sil_values[predictions == i]))
    name_bars.append(f'Cluster {i+1}')

plt.figure(figsize=(11, 4))
plt.grid(visible=True)
plt.bar(name_bars, sil_values, color='skyblue')
plt.title('Comparison between silhouette scores')
plt.ylabel('Silhouette Score')
plt.ylim(0, 0.2)
plt.show()
```

```
Silhouette scores
Whole dataset: 0.1570
Cluster 1:     0.1445
Cluster 2:     0.1383
Cluster 3:     0.1868
```

Figure



Comment the results, also considering the results observed previously (e.g., score graphs, centroids, etc. - max 150 words):

The graph shows that the third cluster is the best one, almost reaching a silhouette score of 0.2, surpassing the average silhouette score of the whole dataset.
The first two clusters instead perform quite similar, lowering the average silhouette score.
In conclusion the cluster built around the centroid 'Cautious humanist' is the most cohesive.